



AWS Certified DevOps Engineer - Professional (DOP-C02): Monitoring and Logging

EventBridge and CloudWatch

EventBridge Overview

Hey, Cloud Gurus. In this lesson, I want to do an overview of Amazon EventBridge. What is EventBridge? Well, think of CloudWatch Events. AWS has renamed CloudWatch Events to EventBridge. And what is provided is some extra functionality, which we will talk about. So in this lesson, we'll take an overview look at EventBridge, and we'll just get into the details of how it works. And then we'll go out to the management console and do a demo. I think a demo is important just because there may be some confusion on CloudWatch Events versus EventBridge. And we'll look at some exam tips. So with all that said, let's take a look at EventBridge. All right, so what is EventBridge? And what does it have to do with CloudWatch? Well, it's just some more functionality to the already existing CloudWatch Events. EventBridge is a serverless event bus that makes it easier to build event-driven applications that scale using events generated from your applications, integrated SaaS applications and AWS services. So what is EventBridge? Some key points. Events has been renamed EventBridge. All this functionality is still there, and your old events still work perfectly. You can create custom events for EventBridge, and we'll take a quick look at it. CloudWatch Events has a default bus, default event bus. So does EventBridge, but you can create a custom event bus. So they share the same API. Again, the old code is good and still valid. EventBridge is fully backward-compatible. Let's see how EventBridge works. So, there are some key elements in EventBridge. There is the event. EventBridge receives an event. Now, the event is tied to a rule. An event is matched to a rule. And if that match occurs, well, we perform some kind of an action. Where do we perform that action? We perform the action on a target. So we're going to have to match an event with a rule that we create or an existing rule. And we specify a target. In this case, I'm specifying CodePipeline. The rule routes the event to an appropriate target. So in this case, let's use an example, a good old example. Hey, our event is going to be the uploading of code to S3. We match that to a rule. So that is our trigger, which starts our CodePipeline. CodePipeline will deploy the new code that was uploaded. Now if you're following along, you can certainly go out and check out these targets that are available in EventBridge. The targets include API Gateway, CloudWatch log group, CodeBuild, CodePipeline, several EC2 actions such as create snapshot, image build or rebuild, reboot instances, stop instances, ECS,



Redshift, Lambda, Kinesis, SNS, SQS, and Systems Manager. But certainly check out that page for more information. But definitely remember the key building blocks. We need an event. We have to match an event to a rule and a rule to a target. Let's go out to the management console and see how we can set up EventBridge events. All right, out at the management console in EventBridge, we're going to create a rule, which will notify us when a snapshot of an EBS volume is created. Now, did a little bit just to speed this along a little bit. So I've created an SNS topic named ebsSnapshot, and I have subscribed to that topic. So when this all goes down, EventBridge will be triggered, send a notification to SNS, and I'm subscribed to that topic, and I should get an email. So let's go ahead and create our rule, and we'll stick with an event pattern. Creation of the snapshot will be our trigger. Click Next. And we have some sample events here Can read through this. It's optional, sometimes helpful. All right, AWS services. Which service? Well, you might initially think EBS, but EBS is essentially part of EC2. Event type, here's where we need to get specific. EBS snapshot notification. And note our event pattern. AWS EC2, again, EBS is within EC2. And let's just keep this general, any event, any result, any snapshot ID. So, if any of our volumes have a snapshot created, that should trigger our event rule. Let's click Next. All right, targets. And again, I've already created the SNS topic. Click on this drop-down. Here's our topic to which I am already subscribed. Let's click Next. Optional tags, Next. Review and create. Create rule. All right, so our rule is in place. Now let's go over to EC2, and don't really need SNS any longer. I'll just use this tab, EC2, EBS. Let's go into Snapshots, and let's create a snapshot. And we have it set up for any, so I'm not going to be real particular. Just select this top volume. Let's create our snapshot. All right, over at EC2, I have one running instance. Let's click on that. Click on the instance ID. Now let's go to Storage. So we have a volume here. Let's click on that. Click the checkbox. Click Actions. Create snapshot. Just give it a description and Create. Let's go over to Snapshots, Pending. All right, didn't take long. We have a snapshot. So, did it work? Well, we created a snapshot. Should have triggered our EventBridge rule. EventBridge notifies the SNS topic, and we should receive an email. And just to confirm, there we go. So we set up that automation. I actually did two. I did one off camera. But there it is. So, our automation is working, notifying us automatically when a snapshot is created. And this is facilitated by our EventBridge rule. All right, so we saw how EventBridge works. Three key elements, the event, the rule, and a target. Just remember the differences between CloudWatch Events and EventBridge. Basically, they're the same thing. CloudWatch Events has been renamed EventBridge. CloudWatch Events is a hamburger. EventBridge is a hamburger with some great toppings, additional functionality. It extends CloudWatch Events. Hasn't replaced it. By that, I mean CloudWatch Events has not and will not be deprecated. Remember, we can create custom application level events and publish them to EventBridge. What are the three key elements in EventBridge? Event. Some good options here, but let's go in sequence. Event, Rule. Where do we want to apply this? What is our target? Those are the three key elements. And one more bonus tip.



Treat EventBridge and CloudWatch Events as the same service because basically they are. And I wouldn't expect a question to mix the two terminologies unless it was trying to illustrate the additional functionality of EventBridge. So that is a look at EventBridge, and we'll have a couple more lessons on EventBridge. So with that, I will end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

CloudWatch Alarms

Hey, Cloud Gurus. Welcome. In this lesson, we're going to talk about CloudWatch Alarms, and we are in the Monitoring and Logging domain in this section. So CloudWatch Alarms certainly play an important part in that domain. Probably in DevOps, and even AWS in general, when we think about CloudWatch Alarms, we think about alarms for auto scaling. So we'll take a look at that, and it's something we should know by now, but certainly review that. We'll look at some metrics to monitor, and then we'll go into a demo and maybe show something that's a little bit underutilized but very helpful from a DevOps perspective. And then we'll check out some exam tips. So with that, let's talk about the CloudWatch Alarms and policies for auto scaling. All right, what triggers our auto scaling policies? Well, you certainly know where this is going. So we have a group of EC2 instances in our auto scaling group. And of course, with auto scaling, we can add new instances and remove instances as well. And this all comes into effect by using a scaling policy, which allows us to scale up and, again, scale down when necessary. But the key to all of this is our CloudWatch alarm. Now, what we choose to scale on depends on our situation. Common one is certainly scaling on CPU utilization, and that is an out-of-the-box metric. Now a custom metric that is very helpful for EC2 instance as well is memory utilization. We can create a custom metric and, again, scale based on memory utilization. So that is our most common use probably of CloudWatch Alarms. And for our auto scaling group, we will create a CloudWatch alarm and scale up if necessary depending on the metric. I'll just say CPU utilization. Let's say CPU utilization exceeds 80%, we scale up. If our resources are underutilized, let's say 10%, we can scale down using, sort of as our trigger, the CloudWatch alarm. The key point to all of this is there is more than one metric for which we can scale out or scale in our auto scaling groups. Here are the auto scaling metrics, group min size, group max size, pending instances. These are auto scaling metrics. And of course, we can get specific to EC2, CPU utilization, disk read ops, disk write ops. And again, memory utilization is a good metric for EC2 instances. And that can be created as a custom metric. Just scroll down a little bit. We have one of our handy light bulbs here. So again, it's important to understand that we have options here with our metrics. CPU utilization is not always the best indicator, and we can create custom metrics such as memory utilization. So let's go out to the management console and do a quick demo. Now, the most common demo, maybe in all of AWS, is having an auto scaling group, creating your alarms for CPU



utilization, scaling up and down based on that CPU utilization. And I've done that in several courses, and I've actually gone to the extent of using a Linux process that would automatically sort of peg the CPU utilization and create that alarm scenario and automatically scale up. So I'm going to show something different, something that is helpful for DevOps engineers and also can be helpful within your personal AWS account. So let's go out to the management console. All right, I'm out at the management console, and I'm in EC2, not CloudWatch. And I just want to show sort of a different way to accomplish things and just do a different demo. I guess the most common demo for CloudWatch Alarms is based on CPU utilization. Well, this demo could possibly save you some money if you're using your personal account or maybe even worse if you're using a company account. But we've all left instances running long after we've needed them, probably multiple times. So let's go into Instances, and we aren't seeing any. We just have to remove this filter And I'll just use this CodeDeployInstance and alarm status. Let's view alarms. The instance has no alarms. All right, so we want this instance to have an alarm. Let's go create an alarm. Go over to CloudWatch, right? Well, we could. But we can create it here as well. So let's click Actions, Monitor and troubleshoot, and you actually have to click that. Manage CloudWatch alarms right from EC2. Let's click that. And notice, we're still in EC2. We weren't jumped over to CloudWatch. All right, so the premise is we don't want an instance running. We forget about it, and it accrues charges. Now we could set a billing alarm, but why not be proactive about it? So let's create an alarm that's going to catch this right away. All right, for notification, SNS topic. And I'm just going to, to save some time, use the topic from the last lesson. Already subscribed to this topic. And if our alarm triggers, we will get an email notification. All right, alarm action, specify the action to take when an alarm is triggered. Let's just toggle that on. Select action. All right. So, we can recover, we can reboot, we can terminate. I'm just going to select Stop. All right, let's create our alarm threshold. And we have a bunch of options here, but I will stick to CPU utilization. And let's do less than and equal, and I'll say 10%. Consecutive period, 5 minutes. So this is a practical example for me. Let's say I create an EC2 instance for a demo, and I complete my demo. I forget to terminate or stop the instance. Well, now I have an alarm set. So, we create the alarm. And again, this instance has stopped. Let's say I started this instance up and did something with it. Probably a demo. I forgot to stop it. Well, after 5 minutes of very low activity, the alarm will trigger, and I will get an email notification. And it's the same email notification from the last demo, so I'm not going to demo that again. But that's the general idea. And just remember, you have many more options rather than just CPU utilization. And also remember, CPU utilization is not always the best alarm to set for utilization of your instance. Sometimes you want to create a custom alarm based on memory utilization. So that's a bit about CloudWatch Alarms. All right, in this lesson, we touched on CloudWatch Alarms. Certainly the most common use of these alarms is for auto scaling groups, scaling out and scaling in. And we also saw how we can keep our EC2 instances under control and manage them and certainly manage the costs of our instances. Let's talk



about a few exam tips. Again, memory usage is not an out-of-the-box metric for EC2. It is certainly very useful and is an option rather than always using CPU utilization. We can create a custom metric for memory usage. Alarms can play a key part in our DevOps automations. Of course, they can be used to trigger auto scaling and SNS notifications, Slack. It's a trigger. Anything that's a trigger we can use with Lambda. And as we just saw in the demo, really managing our EC2 instances straight from the EC2 console and managing costs. So certainly, again, wanted to show something different. But using these alarms to scale our instances, both out and in, is the most common use case for these alarms. So that's a look at CloudWatch Alarms. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

CloudWatch Logs and Metrics

Hey, Cloud Gurus. Welcome. In this lesson, we're going to talk about CloudWatch Logs and Metrics. So we'll start by looking at a diagram and just getting an idea of CloudWatch Metrics and logging. And we'll do a little comparison between the two, what are their differences and sort of their use cases. We'll talk about the CloudWatch Logs agent a bit, and we'll do a demo. And in this demo, we'll see how we can automate moving this data around. It's a lot of data. We need to get it where we can analyze this data, and the demo will cover that. And then we'll wrap up with some exam tips. So with that said, let's take a look at CloudWatch Logs and Metrics. So certainly with monitoring and logging, we are going to be dealing a lot with CloudWatch Metrics, as well as Logs. So we have metrics out of the box with CloudWatch, and then we have our custom metrics, which we've talked about already. So we can create metrics to suit our own needs. And of course, we have our CloudWatch Logs. And we have our logs, we have log events, and we have log streams. So, of course, we can stream this data. And when we think of CloudWatch Logs, the example that comes to mind immediately is log files on EC2. So we install the CloudWatch Logs agent on EC2, and then we can stream those logs over to CloudWatch. Our CloudWatch Metrics are time-ordered set of data points that are published to CloudWatch. And a very recent example, think of the last lesson where we set up a metric for our EC2 instance, so CPU utilization. We can use that metric to shut down our instance or terminate if we want to. Another key point about metrics is that they are regional. So they only exist in the region in which they are created. So CloudWatch is collecting metrics for many different services, and how do we keep this straight? Well, the way this is all kept straight is by using namespaces. And for the exam, just make sure that you understand the syntax of these namespaces. So it's going to be AWS/EBS or ELB or EC2. And in simple terms, a namespace is a container for CloudWatch Metrics. Let's take a look at some terminology. So again, the namespace is used to isolate different application and service metrics. Note the syntax. And we can even create custom namespaces for custom metrics. Now when we talk about logging, some of the terminology log event, the activity that's been reported, log



stream, a sequence of log events from the same source, log group, metric filters, retention policies and the CloudWatch agent. We install this on EC2, and we can automatically publish logs to CloudWatch. Some concepts for CloudWatch Metrics. Again, they're time-ordered data points published to CloudWatch. This only in the region where they are created. They can't be deleted but expire after 15 month. And we've seen this. Services can have multiple metrics. Let's talk about dimensions real quick. Represent a name value pair that uniquely identifies a metric. Examples for EC2, InstanceId, InstanceType, ImageId, AutoScalingGroupName. So we've talked about CloudWatch Logs and Metrics. And let's just see a little bit about how they differ. Metrics publish data points about, for this example, the load balancer and retrieve statistics about data points as an ordered set of time series data. Conversely, the logs will give information regarding requests made to the load balancer. So, another way of stating this is that the logs are capturing information about a specific event, and metrics are a measurement at a point in time for that system. Logs are usually collected anytime an event takes place. Metrics are usually collected at fixed time intervals. All right, let's take a look at another diagram. All right, so we have a load balancer, and we can use CloudWatch to collect information from that load balancer. And key point about this is rules can be applied to these logs to trigger, there's one of our magic words, alerts when certain information is detected, such as out of memory errors. So that's sort of one of those specific events where logs come into play. Of course, we have metrics as well. And one of those metrics and probably the most common is CPU utilization. They can also be used as triggers. And just notice the difference here. So with these metrics, again, we're talking about time-ordered metrics. And I'll use EC2 instance as an example. If you have an EC2 instance, it's going to have a CPU utilization metric. Now that may be 0, 0%, but it's still a metric that we're recording. Whereas with logs, we're not always going to have that out of memory error. It's more of an occurrence. It's not ongoing. When it occurs, we want to capture it and potentially act on it. So a big focus of DevOps and this course is, okay, we have all this data. How do we get it to CloudWatch? Now, we've used EC2 instances as examples, load balancers, and these resources can be a part of an Elastic Beanstalk environment, a CloudFormation stack, an OpsWorks stack. Doesn't matter. We still use sort of the same vehicles to move this data around. And again, for EC2 instances, we can use the CloudWatch Logs agent installed on our instances to capture metrics and logs. And that data can be streamed to CloudWatch. Additionally, we could have some data analysis going on. So getting the data from CloudWatch to our analysis devices, we can use Kinesis and Lambda and Elastic MapReduce. We're moving data around from our instances to CloudWatch and further on down the line if necessary for data analysis. And we've seen metrics and log events can be triggers. And what handles triggers are the same old suspects, Lambda, Kinesis, and they can be used to move this data further along. And let's look at one more thing here. Custom metrics and logging provide flexibility to publish custom data to CloudWatch, beneficial for troubleshooting and creating alarms. Just keep in mind, yes, we have out-of-the-



box metrics, but we can also create custom metrics. Now, just using OpsWorks as an example, again, within our stack and within our layers, we're going to have instances. We handle those the same way we handle standalone instances. We need that CloudWatch agent on the machine, and we're monitoring log files on the instances. And we configure where to send these logs, and then it's just a matter of making sure that our agent is running. And we can handle that in the user data section when we're configuring our EC2 instance. Now focusing on some custom metrics and search and filtering our metrics, four key elements, a filter pattern, metric name, metric namespace. Remember for custom metrics, we can create a custom namespace. And then a metric value good. Gxample would be 404s maybe that we're seeing for our application. So if our app is continually sending 404 errors, we can certainly capture that. We can create a metric and capture that and then act upon it. Part of that action could be, hey, let's take a look at our log files on the instance. So they certainly can tie together and work together. Talked a good bit about metrics and logging, not only in this lesson but other lessons. Let's see about how we move this data around. Go out to the management console and set up an automation for our metrics. All right, out at the management console in CloudWatch, we're going to set up a metric stream using Kinesis Data Firehose. So think about this like a metric pipeline. We're going to have metrics on our EC2 instance, and those metrics will be sent to CloudWatch. From CloudWatch, they can be sent to Kinesis Data Firehose and then ultimately maybe to a data lake environment. So let's click on Streams and create a metric stream. And we have some options here. All right, notice our options here. Quick S3 setup Stream your metrics to an S3 bucket with one click. By default CloudWatch creates the resources for you and streams the metrics using JSON format. Well, what resources are being created? Let's scroll down here. Give us permission to create the following resources in your account, S3 bucket, S3 write role, ultimately a Firehose metric stream. So, just be aware of these resources being created, but that's what we want. So let's click Create metric stream. Your metric stream was successfully created. So everything is set up, and our metrics will be streamed to an S3 bucket. Now, over here, edit actions. Let's say something is just not quite right. Well, we have some options here. We would have to create another metric stream for it to show in the drop-down. Service roles, create a new one. We could change the output format, and this is very important here. Select Metric streams you wish to stream. Stream 21 namespaces. All metrics will start streaming automatically. Well, guess what? We get charged for that. So why don't we scale that down? Select namespaces and metric that you want to stream. I'm going to select Exclude, and we could start excluding. Now, it might be actually easier include, stream only selected metrics and save changes. So we've narrowed down the metrics that we are going to stream to S3. So that's how to set up streaming of EC2 metrics through CloudWatch. Again, think of it as a pipeline, EC2 to CloudWatch, CloudWatch to Kinesis Firehose and, ultimately, to an S3 bucket. All right, so to wrap up, let's jump over to S3. Ah, jumps right out at us. There's the S3 bucket that was created for us, and we'll just have a folder structure here. So we have an EC2 instance



running one instance, but there could be more. And basically, we have it set up so that those metrics are streamed all the way to S3. And if you're following along, just remember to delete these. These streams accumulate charges. Okay, just a couple quick exam tips. And I cover this a lot because you'll definitely see this. How do we get EC2 log data to CloudWatch? We got I have to install the CloudWatch agent on the EC2 instance. Typically, we install that in the user data of the instance, and then it installs when we spin up the instance. Another thing you probably see is some CloudWatch namespace syntax. Really easy so just know that. And the only real outlier to this is if you create a custom metric, you're going to have a custom namespace. So that's a look at CloudWatch Metrics and Logs. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Automation with EventBridge and SNS - Demo

[Autogenerated] all right, so I'm at the Eventbridge console, and let's go into rules. So in a previous lesson, I created an event bridge rule to create an EBS snapshot every 3 minutes, and what I've done is disable that because I didn't want a snapshot being created every 3 minutes. I could have, of course, deleted it, but I decided to use it for this lesson. So we're going to use this rule. But first, let's go over to SNS. And let's go into topics. And create a topic. And we're not doing much here, so I'll just go with a standard topic, give my topic a name. All of this other stuff is optional, certainly important, but optional, and we just want to create a quick topic. Now, with that, we need to create a subscription. And we just want email. But as always check out these drop downs. It's a good study tool know the options available. And the endpoint is an email address that can receive notifications from SNS. Let's go ahead and create the subscription. Now, offline, and I'm just not showing it on the recording, I received an email to confirm the subscription. So I went ahead and did that. I'm now subscribed to this SNS topic, and anytime a snapshot is created for the volume we specified, I should get an email. So let's go over to Event bridge. And remember our rule. And right now we have one target, create a snapshot. So every 3 minutes, it's going to create a snapshot for us. Well, guess what? Let's edit this. We can add additional targets. And that's sort of why I wanted to reuse this example. Just wanted to show that yes, we can have multiple targets from one event. So, of course, our 2nd target is going to be our SNS topic that we created. Let's go ahead and update that. Now we have one problem left and it's probably jumping out and screaming at you in red. Our row is still disabled, so let's go ahead and enable that. There we go. all right, now, I'm going to switch the screen. I'm going to show my email, and we'll see that I have received an email notifying me of action, specifically the creation of a snapshot. all right, so I'm into my email and I actually drilled down into the message that I received. Not, scheduled event, AWS events. Here's the key. EventBridge EBS demo. So our notification from EventBridge is working through SNS to my email. Now, if you follow along with



this demonstration, please remember to either delete or disable that event. Remember, it's creating an EBS snapshot every 3 minutes, and we certainly don't want all those snapshots building up in our account. So please remember to either delete or disable the event.

Supporting Services

CloudTrail Overview

Hey folks, welcome. So in this lesson, we're going to take a look at AWS CloudTrail. And AWS CloudTrail is a service that offers governance compliance and risk auditing of your AWS account. So we talk about keywords on the DevOps Pro exam, and one of the keywords auditing. It's not a one-to-one match with CloudTrail. But if you see auditing, certainly have CloudTrail in the picture as a consideration. So we'll just take an overview look at CloudTrail, and we'll also check out a CloudTrail log. Get into the log file and see what it looks like. We'll look at the workflow of CloudTrail, how it all fits together with the rest of our AWS services. And we'll just take a look at a couple of test cases. Again, it sort of ties everything together. And we'll go over some exam tips. So that said, to quote my robot friend, let's learn about CloudTrail. So in layman's terms, CloudTrail is a tool that you can use to make sure bad actors aren't doing bad things in your AWS account. And when I say bad actors, that extends beyond the cast of Sharknado. So let's start tying this all together. So how can we work with and access CloudTrail? Well, we can do it through the management console. We can do it with AWS tools and SDKs, basically programmatically. I always fall back on Java. We can make calls in our Java code using SDKs to CloudTrail. And we can also use the AWS CLI. So what services interact with CloudTrail? A better question would probably be what services don't interact with CloudTrail? There are a lot of services, and I'm showing just a few, a few of the more common ones that we would encounter. Let's say we're setting up a VPC to house our EC2 instances and auto scaling in groups. But all of these services interact with CloudTrail. CloudTrail is keeping an eye on all of the services and logging key interactions that these services make. Now, I do want to focus on this middle icon here, AWS Tools and SDKs. So from my program, maybe a Java program or C# or whatever, how are we interacting with AWS? Well, we're probably making API calls. And what CloudTrail is doing is it's keeping a history of our API calls and related events for our AWS account. So for the services that support CloudTrail, CloudTrail identifies which users and accounts called AWS and what API calls were made. And one of the things that's captured is the source IP address calls were made from and also when the calls were made. and CloudTrail is per AWS account and enabled per region for all the services supporting it. So think about that. When we're capturing these API calls, the API call history. With these API calls captured, we can perform security analysis, resource change



tracking, and compliance auditing. There's that word again. So your two key terms to remember for CloudTrail are API and auditing. So, CloudTrail is capturing these log files. Where is it storing the log files? And by now, you can probably guess that, S3. So basically, CloudTrail is monitoring our entire account across all of our AWS infrastructure, captures these API calls, who made the call, when they made the call, stores those in S3. Now, of course, we can then review these logs and act upon them. And of course, by now, you can probably guess, we can set up automations. Our normal automation tools, EventBridge aka CloudWatch Events, and the usual suspects, SNS notifications. We could trigger a Lambda or a Slack message. Or we can trigger a Lambda to grab the logs and perform or send them somewhere for analysis. So let's talk about a CloudTrail workflow. So again, it's tracking user activity and detects unusual API activity. It's going to detect all the API activity but, of course, capture anything unusual. And activity by our AWS services are CloudTrail events. And again, CloudTrail delivers the events to the CloudTrail console, S3 buckets, and optionally CloudWatch Logs. And again, we can set up automation. We can use CloudWatch Events. We can also use CloudWatch alarms and take action when important events are detected. So the point of this is, yes, we can capture logs and just store them for compliance purposes. Maybe we have an audit coming up in 6 months. But we can also automate this and act on the information in these logs almost immediately just by setting up some automations. And of course, we can view recent events in a CloudTrail console. We can scan through the logs. But that can be automated as well. And we can analyze our log files with Amazon Athena. So there's a lot of different ways we can go with this, but we see some repetitive patterns here where we are automating things with services such as EventBridge events and Lambda. So those patterns repeat. All right, let's talk about a test case. So we have multiple AWS accounts. Can we consolidate our log files into one place? All right. First of all, let's get the easy part out of the way. Where's that one place? That one place is going to be an S3 bucket. And just think about the features of an S3 bucket. S3 buckets are regional. So yeah, we can absolutely do that. It's just a matter of how do we set this up. So we have— Let's say we have three AWS accounts. And we can send our CloudTrail logs for each of these accounts to the same S3 bucket. Now, as you might expect, there's a little bit of permissions that need to be set up. So, first of all, we set the S3 bucket as the target for CloudTrail in each account. That's easy enough. And we also create a bucket policy granting cross-account access. So, we store these logs from each of our accounts. In the S3 bucket, we have this policy set up. And remember, when we configure or when we create an S3 bucket, let's say we're in account 2 and we create this S3 bucket. This cross-account access could be applied to account 1 and account 3. And lo and behold, everybody has access to this bucket. So that is pretty straightforward. Let's talk about another test case. All right, we have an upcoming audit. Now one thing about an audit, we want to guarantee the integrity of these log files. So we're storing our CloudTrail logs, our CloudTrail log files, in an S3 bucket. What's to keep someone from tampering with these log files? Of course, we know CloudTrail is being used by our three



accounts. I'll just continue with those three accounts from the previous test case, storing it in S3. Now I'm showing sort of a policy over here by CloudTrail, and just think about that for a minute. What are we doing here? How can we make sure our log files on S3 are not tampered with? Wouldn't we put something on S3? Well, we already have a bucket policy on S3. These accounts can access S3. But it wouldn't be unheard of from someone within a company, someone within one of these accounts, tampering with those log files. So what do we do? We have to do something at the CloudTrail end of this. And if you know this, that's great. If you don't, no big deal. You know it now. We have to enable log file validation on CloudTrail, and that's how we would guarantee the integrity of our CloudTrail logs. And this log file validation lets us know if a log file has been modified, deleted, or remained unchanged after it has been delivered. So that's a key point and a bit tricky. You would think that you would set it up on the S3 bucket, but you actually set it up at the source on CloudTrail. So we've talked about a log file. Let's take a look at a CloudTrail log file. All right, so we've seen a bit of CloudTrail already, but let's just take a look at it in a little bit more detail. Look at some examples. And first, let's start with the CloudTrail log file name format. Notice, account ID, CloudTrail region name, datetime, a unique string to differentiate it, and then file name format. And remember, oftentimes, CloudTrail logs are saved for audits. So we don't want to overwrite these. That's where the unique string comes into play. Let's look at a couple of examples. We have a user named Mateo, ran `aws ec2 start-instances`. Where did he run that from? CLI? Obviously called `StartInstances`. And that action is logged in CloudTrail. So, CloudTrail is going to tell us who did what and when they did it. Very crucial information certainly for audits and maybe if you're trying to track down some suspicious activity. So anyway, we have an IAM user. And of course, that was Mateo. Account ID, access key, username, time they did the action. Here's the event information. EC2 start instance in `us-east-1`. Source IP address. Two instances were started. So these are generally pretty readable. Again, it's who made the call, what did they do, where did they make the call from, that can be important, and what time did they do it? And this is in a JSON format. All right, let's take a look at another one. And this time a user Nikki is from the CLI, `aws ec2 stop-instances`. Same information. Who, what time, where from, and to what? The EC2 instances are stopped. So that is the request to stop these instances, and then we get a response. And think about in the management console. When we click to stop an instance, we get back state information such as stopping and ultimately stopped. All right, let's look at one more. So we have a user named Arnav, and Arnav is creating a key pair. ARN of Arnav, account ID, access key. Remember, it's a CLI command. Username. When did this happen? What's the event? `CreateKeyPair`. Where did the request originate? So if you spend any amount of time in AWS, you'll spend your share of time looking at some JSON code, maybe in log files or in IAM policies. And of course, CloudFormation. Pretty easy to read. With CloudTrail, who did what, when, and where did they do it from are the key pieces of information, and there's not much more to it. So in this lesson, we took a look at CloudTrail, how we use it, the



use cases, how we access it. Really key words for CloudTrail. Think about audits and/or API calls, especially API calls. So let's take a look at some tips. Of course, we're storing our log files in S3. We can work with CloudTrail using a management console, CLI, and SDKs programmatically. Think about what programs do. They make API calls, and CloudTrail tracks that usage. Really important to understand the use cases for CloudTrail, saving our log files for potential audits or for troubleshooting or security issues. And for more immediate analysis, we can set up automations, and we can send these logs to Athena for analysis. Again, more general use cases, auditing, security monitoring, and operational troubleshooting. Now if our auditing is really out long-term, we can ship the logs from S3 to Glacier for cheaper storage. In scenario questions, CloudTrail is often grouped with distractors such as CloudWatch, SNS, SQS, and it's all about knowing the use cases for each. Now CloudWatch is more closely associated. Of course, CloudWatch has logging, but logging of API calls is CloudTrail. So that's a look at AWS CloudTrail. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Kinesis Overview

Hey, Cloud Gurus. In this lesson, we're going to take an overview look at AWS Kinesis. And I say overview. There are a couple more lessons on Kinesis coming up in which we will go much deeper. So what can Kinesis do for us? Well, Kinesis makes it easy to collect, process, and analyze data, and it does so in real time. And that key term is really important for Kinesis. If you see real time on the DevOps Pro exam, start thinking Kinesis. There are other services that can deliver real time such as Lambda and CloudWatch Events, but it's typically associated with Kinesis. So in this lesson, we'll take an overview look at Kinesis and answer the question, what is Kinesis? And we'll look at each of the four flavors of Kinesis and define what they can do for us. And we'll look at a couple of use cases just to tie the concepts together. And then finally, we'll look at some exam tips. Again, this is a quick lesson. We'll go deeper in upcoming lessons. So let's check out Amazon Kinesis. What is Kinesis? Well, we've sort of answered that. It collects, processes, and analyzes data in real time, and that key term, again, is super critical. There are actually four flavors of Kinesis, one which captures data streams and another which captures video streams. And again, just focus on this key term when you're thinking Kinesis. Always associate it with Kinesis, and you can start eliminating some of the incorrect answers in a scenario question. How many different varieties are there? Well, I kind of gave that away already. There are four. And it's critical to understand the use cases for each of those. So again, under the Kinesis umbrella, there are four different services, Kinesis Video Streams, Kinesis Data Firehose, Kinesis Data Streams, and Kinesis Data Analytics. All right, let's just work from left to right, and let's talk about Kinesis Video Streams. And as his name implies, it's easy to stream video from connected devices. And we'll stream it to AWS for analytics, machine learning, playback, and other processing. We can



stream video from millions of devices. Create real-time viewing and video-enabled applications. Now just one potential use case would be, and I have it sort of highlighted above, a smart home or think about just security for companies, security for retailers, or maybe large sporting events. So for security purposes, a sports team could have video devices in their huge parking lot with thousands of people, and it could be hooked up to Kinesis Video Streams, and Video Streams capture that information. And again, it could be used for security purposes. And let's just keep on that security aspect here for a minute. Kinesis Video Streams, it could be streamed to Amazon Rekognition. And Amazon Rekognition could focus on a potential assailant's face. And that's how we can get immediate feedback from a security standpoint. So the video streams can ingest this video data. It can durably store this data, and it also can encrypt and index these streams and prepare them for some analytics or machine learning. So I suppose if there's one thing for video streams you want to associate Kinesis with, it would be security. That's another key word that you might want to start thinking about. And certainly, if it is video, start thinking about Kinesis Video Streams. All right, what about Kinesis Data Firehose? What can we do here? So, we can load streaming data into data stores, data lakes, analytics tools. Think about ETL, extract, transform, load. So our Firehose can capture data, transform it, and it can load it into such storage locations as S3, Redshift, Elasticsearch. So we're getting this to our back-end analytics in near real time. And I have listed IoT analytics. So we could have millions of IoT devices, and we capture this data and stream it and have it analyzed in near real time. Now you might already be thinking, hey, wait a second. We could be streaming video using Kinesis Video Streams to analytics tools. And the answer, if you're thinking this, is yes. We can stream video into Data Firehose. Then Data Firehose can perform necessary operations and/or move it on to one of the analytic services mentioned. All right, what about Kinesis data Streams? Well, Data Streams is scalable and durable. It's a real-time data streaming service. So it can capture very large amounts of data, gigabytes per second. And this can come from website clickstreams, database event streams, financial transactions. And this data, when it's collected, it's available in milliseconds. And think about real-time dashboards, real-time anomaly detection, dynamic pricing, and I'm listing ordering of records. We could stream related records to the same record processor. Think about Amazon Elastic MapReduce. So with data streams, some keywords, of course, real time but capturing and streaming large amounts of data. Finally, let's talk about Kinesis Data Analytics. So with Kinesis Data Analytics, it's the easiest way to analyze streaming data and gain actionable insights. And the idea is to respond to our business and customer needs in real time. So some use cases, streaming ETL, extract, transform, load for IoT, real-time log analytics with SQL. So with ETL, that's a database term. Think about we're grabbing a bunch of data. And ultimately, we're streaming that data. It's going to end up in a database. What about log analytics? Think of log files. So we stream that using data analytics. And with SQL, obviously, we can do some queries and analyze this data. So again, we're seeing some overlap here in services. And in fact, we can chain some of



these services together. So always keep that in mind. Let's take a look at some use cases. All right, a top secret government agency needs to stream security camera video to Amazon Rekognition to allow employees off hours access to the facility. Now we know where this is going. What service can they use to stream the data to Rekognition? Well, that's pretty easy. But with Kinesis, it's kind of about the fundamentals. If you know the use cases for each of the four services, you're in good shape. So we have our security camera video, and we need to get that data to Amazon Rekognition. That's a pretty easy one. Amazon Kinesis Video Streams. All right, Formula 1 racing. They have hundreds of IoT sensors on a car. So we need a service to ingest and store the sensor data, also trigger a Lambda function that will quickly analyze and make decisions based on that data. So what service can we use? All right, so we have Amazon IoT. We're basically triggering a Lambda function that's going to make some decisions based on our data. Now that's kind of vague. But as we know, Lambda Functions can do whatever we can code them to do. So what do we need here? Amazon Kinesis Data Streams. So in real time, it can ingest and store that sensor data. It's going to trigger a Lambda function. Lambda function is going to be passed some information from our Kinesis Data Streams and make some decisions based on that. I suppose with thinking about a Formula 1 racing car, it could be related to fuel, fuel level of the car. It could be related to, think about tires, the tire wear. Those racing cars go through tires very quickly. It could be related to engine temperature, all sorts of things, brake fluid levels. So with IoT and Data Streams and then, of course, Lambda, we can pretty much do anything we need as far as acting upon data from a vehicle. All right, so I want to give a quick overview of Kinesis and the four flavors of Kinesis. First and foremost, if a scenario uses a term real time, start thinking Kinesis as part of the solution. Then it becomes a matter of knowing the use cases. Which offering of Kinesis or maybe offerings? Maybe we need to chain two offerings together. So know the use cases. Streaming video data, that one's pretty obvious. Streaming and manipulation of data such as ordering of records, Kinesis Data Streams. Streaming ETL data or real-time log analytics, Kinesis Data Analytics. And Firehose, large amounts of data, streaming that into a data lake or a warehouse. So that's a look at Amazon Kinesis. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Kinesis Data Streams with Lambda - Demo

Hey, Cloud Gurus. Welcome. So in this lesson, we're going to look at using Kinesis Data Streams with AWS Lambda. So we'll start the lesson taking a look at the architecture of this scenario we're going to look at, and we'll take a look at the steps involved. Then we're going to have two files. We'll have a Lambda function, as well as an input file, which is going to simulate a Kinesis Data Stream. And then we'll go ahead and go out to the command line interface and execute this scenario. And we'll come back and just go over some exam tips to close out the



lesson. We have a lot of ground to cover, so let's just go ahead and get started. All right, so in this lesson, we're going to take a look at Kinesis Data Streams as both a receiver and a sender of data. So we're going to have a custom application. Basically, it's just going to be creating some data for our Kinesis Data Stream. This data stream is going to accept the data, and it's going to trigger a Lambda function. You'll see some of our common patterns here. First of all, what do we always need? We need an execution role. We're going to have some functions acting on our behalf. And ultimately, our Lambda function is going to stream information into CloudWatch Logs. So some real commonalities here. Always think about our execution role from IAM and notice how we're chaining services together to create the automation that we want. Sort of an overview but let's take a look at some specific steps we're going to need to execute. So first of all, again, our custom app is going to write records to our Kinesis Data Streams. AWS Lambda is going to pull the stream. And when it detects new records in the stream, it's going to invoke our Lambda function. And again, we're going to have to create an execution role to allow Lambda to perform actions on our behalf. Now the majority of this lesson is going to take place at the command line interface. So I'm going to switch the screen and go out to the command line interface. All right, so I'm out at the management console in CloudShell. And however you get to the CLI, whether it's CloudShell or SSH or Instance Connect, you, of course, need the AWS CLI. And you most likely have it. And with this command, `aws version`, you can verify that. Now when we're working with Lambda, just think about what we need. Absolutely, to work with Lambda and take it a step further, our Lambda function needs permission to access AWS resources. So what we need is an execution role. So I have an IAM tab open. I'm going to Roles. And I've already created a role just to save a little bit of time, `lambda-kinesis-role`. Notice the policy, `AWSLambdaKinesisExecutionRole`. And most importantly, our Lambda function has some access to Kinesis and also CloudWatch Logs. So that's the execution role for our Lambda function. Let's go ahead and create the function. And we could do this from the management console, but we can also do it from the CLI. This function is going to process the Kinesis messages and ultimately log those to CloudWatch Logs. So let's first create a directory and `cd` into that directory. I'm going to use `vim`. Create a file named `index.js`. Now we don't need to know in-depth JavaScript or Node.js for the DevOps Pro exam. What we need to know about this Lambda function is that the function grabs the event ID and event data of the Kinesis record and logs it to CloudWatch Logs. And I'll include this code in the lesson material. And to save this file, I'll just press `Escape :wq` bang or exclamation and quickly list the contents. There's my `index.js`. All right, how do we deploy code? Well, we need to create a deployment package. Zip this up. We'll call it `function.zip`. And what are we zipping up? Our `index.js`. Now we're not done yet. We actually have to, from the command line, create the Lambda function. So let's have a look at the `lambda create-function` command. And it just went ahead and ran on me. I got some JSON back. Always a good sign, but let's run through this command. So, `aws lambda create-function`, function name,



ProcessKinesisRecords. Remember this name. Note, it's referring to the zip file we created, index.handler in our index.js. Here's the runtime Node.js. Now, the role ARN, we have to have that execution role. There's our role ARN. And this went ahead and ran. Let's go over to Lambda. Trust but verify. We supposedly created a Lambda function just now. Ah, there it is, ProcessKinesisRecords, 2 minutes ago. And of course, from the CLI, not seeing any error message was a good sign as well. All right, so I'm just going to clear the screen. Now we have some work to do with Kinesis. So we're creating a Kinesis stream, `aws kinesis`. We set the context of our CLI command, `create-stream`, `stream-name`. We give it a name. How many shards? We're just testing here. We don't have to get carried away with shards. So let's create our stream. All right, no news is almost always good news, but let's verify that. `Aws kinesis describe-stream`. What stream? Well, the one we just created, `lambda-stream`. Okay, looks promising. Now, the added bonus is we get the stream ARN from that command. So I copy that to my clipboard. I can just quit out of that. Clear the screen. Okay, so we have a Lambda function that we want to be triggered by our Kinesis stream. Now, do we have any linkage between the two? And that answer is no. So we have to map our event source, which in this case is our Kinesis stream to our Lambda function. Remember those event sources. All right, that just runs, but let me walk through this, `aws lambda create-event-source-mapping`. What are we mapping to? Well, our function. And our event source is our Kinesis stream. Here's the ARN for that. It just gives a batch size and the starting position, latest. In other words, when there's a new record, it's going to start from that latest position and not just sending over everything all at once. With the CLI commands, getting back some JSON is almost always a good sign. Notice this ID UUID. That's an event source mapping. Let's go over to Lambda. I'm in Event source mappings. Let me refresh that. Ah, there's our event source mapping. Remember this UUID? There it is. So now we have a Lambda function, we have a Kinesis stream, and we have an event source mapping. Our event source Kinesis is mapped to our Lambda function. Let's test this out. Just clear to make a little room, `aws kinesis put-record stream-name lambda-stream` and just a little message. Let's run that. Ah, got back a shard ID and a sequence number. That looks promising. So our Lambda function is grabbing information from our Kinesis stream and putting that in CloudWatch Logs. Let's go over to CloudWatch Logs. All right, over at CloudWatch, I go into Log groups, and let's find our log group, `LambdaProcessKinesisRecords`. Now, let's bounce over back to the command line. Let's just remember 4965. Back over to CloudWatch. Here's our log group, log stream. Just a few minutes ago, very promising. Ah, there's our shard ID, 4965. So that worked. From the command line interface, we created a Lambda function, and we also created a Kinesis stream. The Kinesis stream is our event source. We created an event source mapping between our Kinesis stream and our Lambda function, and we were good to go. Our Lambda function is now logging Kinesis records to CloudWatch Logs. So in this lesson, we looked at how to stream data from Kinesis to trigger a Lambda function. And of course, Lambda can do anything under the sun that we program it to do. In this case, it streamed



data on to CloudWatch Logs. Now if you follow along with the lesson, you can go ahead over to EC2 and either stop your EC2 instance or terminate the instance. And remember, if you created access keys, go into IAM and delete those access keys. Always create the keys when you need them and just delete them as soon as you're done. So let's look quickly at some exam tips. Kinesis often acts as both an input and an output in scenarios. Our streams receive data, manipulate the data, and then outputs the data, in this case, to a Lambda function. We triggered a function. The output could also be a data lake or SQL. Kinesis triggers our Lambda function. Remember the steps to execute a Lambda function. Most importantly, that execution role, Kinesis, CloudWatch, and Lambda. So as we see, they can be used to automate our processes. They can work together for this automation and understand the use cases for each. We're seeing a lot of repetition now with our automations, and that's good. That'll ingrain these concepts. So with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

X-Ray Concepts

Hey, Cloud Gurus. Welcome. In this lesson, X-Ray Concepts, we're going to talk about the AWS service AWS X-Ray. So in the lesson, we're going to take an overview and, first of all, sort of define the problem that X-Ray is solving for us and how X-Ray works. And we'll get into color-coding with X-Ray. Just bear with me on that. I don't want to give that away just yet. And we'll take a look at the workflow for X-Ray. And then finally, some exam tips. So let's go ahead and talk about X-Ray, what it can do for us, what kind of problems it can solve for us with our distributed applications. Let's talk a little bit about the environment we're working in, the cloud environment. We're dealing with microservices. And we have a single application. With microservices, this can be composed of loosely coupled and independently deployable smaller components. And there are many advantages to this, but it does complicate troubleshooting and just performance tuning our applications. There are more moving parts, and it's just a little bit harder to trace. And that's where X-Ray comes into play. All right, so what is X-Ray? X-Ray is a service that collects data about requests that our application serves, and then it just provides tools that we can view, filter, and gain insights into the data and requests going through our application. So again, it can provide troubleshooting capabilities, performance tuning capabilities, and a great overall picture of what's going on as a request travels through our application. Now how does this work? So we have our application code. And what happens is if we're working with X-Ray, we have an X-Ray daemon that's collecting what we're calling trace data from our application. Now the daemon can collect this. It can actually queue it up. This doesn't have to be instantaneous. Ultimately, our trace data is forwarded to X-Ray, and X-Ray is going to provide us a graphical depiction of those requests traveling through our application, touching our microservices and different services, and it's going to provide, first of all, just a little bit of



performance feedback. And momentarily, we see these circles for each of the stops. Well, they can be color coded. And depending on the color X-Ray provides for us, we know, okay, this is good or this is an error, this is a problem, this is slow. So it's really an impressive service. Now, one other thing. Okay, application code going through SDK, but we can have other clients, scripts, and tools input from the CLI. Of course, scripts can be built through the CLI and run. But the daemon, the X-Ray daemon will collect that trace data as well. So it doesn't have to come just from one source. Doesn't have to come exclusively from that application code. It's going to collect all of this data and forward it to X-Ray. X-Ray is looking at our application from end to end and providing this information back. So let's talk about a few key points for X-Ray. X-Ray collects data and that said daemon about requests from our application servers and provide tools to view, filter, and gain insights to identify issues, so errors, and opportunities for optimization. X-Ray uses that trace data, and it's going to generate that service graph that we saw. Again, we'll talk about the color coding momentarily. We can identify bottlenecks, latency spikes, and other issues and work to correct those. So we have detailed information about the request and response but also the calls our app makes to downstream AWS resources, microservices, databases and HTTP web API. So, basically, anything our application is touching, microservices, database, and anything else, we can get X-Ray to evaluate that for us. All right, let's talk about color coding. So, these graphs— And this is sort of just a small example graph. They can certainly have more components than this. But they're going to come back ideally all green. Let's look at our color codes. Green is a successful call. Red, just have an example here, server faults. Yellow, client errors. Purple, throttling errors. So basically, anything that is not green we want to work to improve that. So this is just one gone totally bad, all red. And then we decide it's going to provide more information. So here's a Lambda function. And ultimately, Lambda is making a call to DynamoDB. So this starts with our app. Our app kicks off a Lambda function, calls DynamoDB. X-Ray traces the whole way through. It's sort of like a performance review. Also troubleshooting. Red, specifically, that's an error. One other thing about these, notice the response times here. So again, errors, response times, throttling. Ideally, our graph is all green. All right, let's talk a little bit about the workflow. And we already have talked about it really, but let's review. We generate traffic. Our app does. Send the data to X-Ray. That's that X-Ray daemon. It's really the key to this. So we have that X-Ray daemon running. The X-Ray daemon actually passes our trace data to X-Ray. X-Ray produces our service graph. We view the traces and troubleshoot and fine-tune as needed. X-Ray is one of those services. You may run into it on the exam. And typically, doesn't go real deep on X-Ray. Certainly know what it is, what it does, and how it works. And that's what I provided in this lesson. With that said, let's check out some exam tips. Just know the use cases for X-Ray. CloudWatch and CloudTrail could be common distractors. Just remember what it does. Helps developers analyze and debug distributed applications. So, key terms may be distributed applications. Most definitely microservices. Another common key term, end to end. X-Ray gives you



the ability to trace and troubleshoot your distributed app from end to end. That is really a common key term. X-Ray can pinpoint issues, as well as present opportunities for optimization. That goes back to the color coding. You either have an error, maybe a throttling issue. X-Ray trace is a set of data points that share the same trace ID. That's what that daemon is gathering from our app. An X-Ray segment encapsulates all the data points for a single component. And that's where we get into our graph. So that's a look at AWS X-Ray and X-Ray concepts. Thank you for joining me in this lesson. I'll see you in the next lesson.

Amazon OpenSearch

Hey, Cloud Gurus. Welcome. So in this lesson, we're going to talk about AWS OpenSearch. So in this lesson, we'll take an overview look at AWS OpenSearch, and we'll talk about some of the inputs and outputs, inputs to OpenSearch, output from OpenSearch. Where does it go? And then we'll do a quick demo. We'll create an OpenSearch domain, and then we'll attach a CloudWatch log group so that we can stream logs to OpenSearch. Get our log data in OpenSearch, and then we can perform a lot of actions from OpenSearch, accessing that data. Probably heard of the ELK stack. Well, it was Elasticsearch, Logstash, and Kibana. And finally, we'll wrap up with some exam tips. So let's go ahead and check out AWS OpenSearch. All right. So first of all, what is AWS OpenSearch? So OpenSearch is a managed service, and it makes it easy to deploy, operate, and scale OpenSearch clusters in the AWS cloud. Now as this name implies, we're going to search on data and perform analytics, and it is often linked with Logstash and Kibana. Think of Kibana as a dashboard. It's a popular open-source visualization tool designed to work with Elasticsearch. And I'm talking about the OpenSearch version of Elasticsearch, not the AWS now known as OpenSearch, Logstash. Just think about that name literally. Logs and we're stashing them somewhere. We want to store them somewhere. And more importantly, upload the data we're storing in Logstash into our OpenSearch domain. So we're getting data into OpenSearch with Logstash and with Kibana. There's a great dashboard we can use to visualize this data. Now a key point about OpenSearch is that is an AWS-managed service and comes with many of the benefits we've seen with AWS-managed services. So let's talk about some of the use cases for OpenSearch. Log Analytics, and we're going to see part of this in the demo. Again, we're going to stream log files from CloudWatch into OpenSearch. And we could subsequently perform analytics on this data using some of our favorite analytics tools. Time series analysis. We could be dealing with a large amount of data here. ITops monitoring. And again, thinking of CloudWatch Logs, well, we can get data from our servers to CloudWatch Logs and then, again, stream those to OpenSearch. Clickstream Analysis. Oftentimes, when we think of Clickstream Analysis, we think of Kinesis. And yes, absolutely, we could use Kinesis to stream data into OpenSearch. Already seen a bit, but let's take a look at some of the inputs to OpenSearch and the outputs from



OpenSearch. Some of our inputs could be log files, CloudWatch Logs being one but not the only. It could be messages, metrics, configuration info, maybe from servers, and documents. So large amounts of data that we want to curate somewhere and search on. And we just have to use various methods that we're very familiar with to get this data into OpenSearch. And again, in the demo, we're going to stream log files from CloudWatch into OpenSearch. Let's take a look at some outputs. Application monitoring. Security and event management. Obviously search. Infrastructure monitoring. Now some of the tools for this, again, Kibana provides a very robust dashboard that can much of what we're talking about in these outputs. The analogy I would make is let's say we have a database and we just have a very robust user interface to display our data from the database. This is a similar scenario. OpenSearch stores all this information, and Kibana, with that robust dashboard, analyzes and displays that data for us. I'd encourage you to check out Kibana. It's not necessary to know it in depth for the test, but certainly know that it can be a key tool in working with OpenSearch. Again, going back to the ELK stack, Kibana was the K, Logstash is the L, Elasticsearch, now known as OpenSearch, is the E in the ELK stack. All right, so we have a good idea about OpenSearch. Now, let's go out to the management console. We're going to create an OpenSearch domain, and we're also to link a CloudWatch log group to our OpenSearch domain, which will enable us to stream log files directly into OpenSearch. So that's sort of the input to OpenSearch. We're not going to, in the demo, cover the outputs, but we certainly know what we can do on the output side. All right, out at the management console in OpenSearch, we want to create a domain. And you can think of a domain as a cluster of servers, and we can dictate how many servers are in that cluster. Let's go ahead and create a domain. Give our domain a name. All right, creation method. We have easy create. Create an OpenSearch domain using multi AZ with standby for high availability. You can change some configure options after the domain is created. And then there's standard create. You can choose all configuration options, including availability zones, instances, and storage. It's sort of like Elastic Beanstalk where you can get as involved with your infrastructure creation as you'd like to. But the default is easy create. No VPC access. And certainly for security, that is recommended. I'm just doing a demo. And because I'm creating a cluster of instances, I'm deleting this right after the demo so as not to accrue charges. All right, moving down. And here's a look at what is being created. So there's a lot going on here. I'll go ahead and click Create. Wow. Missed this upon the first scan. Fine-grained access control. Numerous features to keep your data secure. And what I want to do here is create a master user. And I just need to make sure I get that right, uppercase, lowercase, and a special character. Now we can create. OpenSearch domain is being created. We have a bit going on here. This might take a few minutes. And you get a status and percentage done here. We could view details. Lots of tabs here. So let's click on View details just to get a deeper idea of what's going on. So in a nutshell, we validate, create the new environment, provision the nodes, and then set up the traffic routing through the new nodes. All right, so cluster



creation is complete. Cluster health is green. It took about 20 minutes. It's a fairly hefty cluster, three nodes. Here are some of the other information. Now the coverage for OpenSearch on the exam is not too heavy. You just need to know what it does and how it's set up. And sort of in a nutshell, we want to stream data, log files, metrics, configuration information into OpenSearch. OpenSearch has dashboards and Kibana to where we can analyze this data quickly. And that's pretty much the gist of what we need to know about OpenSearch. So I'm going to wrap up by going over to CloudWatch and seeing how to link up a log group with OpenSearch. All right, so I'm in CloudWatch. I go to Log groups. Here's that log group we used in a recent lesson. Let's go to Actions, and we want subscription filters. We need to click this and create OpenSearch subscription filter. And select for this account. Only have one cluster, select it. Execution role, and I create an execution role, Lambda for OpenSearch. Basically, a role which gives Lambda full permission to OpenSearch. Log format of Other. That's the catch-all. I'll keep that. But as always, study your drop-downs and see what's available. Don't need a pattern, but let's give it a name. We could perform a test, but I'll just click Start streaming. Up top, you can view the progress. Okay, so to wrap up, we created an OpenSearch domain. We can stream logs to that domain, which we are doing. And then we can perform detailed searches within OpenSearch. All right, let's take a look at some exam tips. Open-source version outside of AWS. We are working with the AWS offering of Elasticsearch. Remember though, it's been renamed to OpenSearch. What's the difference between this open-source version and AWS's? Well, AWS version is managed service, will handle servers for us. We saw that. We just had to specify what we wanted. AWS will create and subsequently manage those servers for us. There's that key term, real-time analytics. Yes, OpenSearch can work with Kinesis and Lambda. We actually just saw that. We had a log group where Kinesis, previous lesson was sending data into CloudWatch. CloudWatch using Lambda can then send log files to OpenSearch, all in real time. Remember, Kibana, remember our ELK stack. Kibana has great dashboard and analytic capabilities. Logstash is going to move large amounts of data around for us. It can store it as well. Key use cases, Log Analytics. We just saw that. We didn't do the analysis part, but we got the logs into OpenSearch to prepare for analytics. Time series analysis, ITOps, and search engine capabilities. So that's a look at AWS OpenSearch. And one last time, I'll emphasize this name change just took place in September 2021. OpenSearch previously known as Elasticsearch. Please write that down and remember that. Another way it may be referred to is Amazon ES for Elasticsearch. So you might see that yet on the exams. Write that down, please. So with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Tagging and Resource Groups

Hey, Cloud Gurus. Welcome. So, in this lesson, we're going to talk about tagging and resource groups. There's a lot we can do with tagging and resource



groups. But of course, this course focuses on DevOps, and we can certainly use tagging and resource groups for automations. In other words, we can choose to deploy, use our automations and deploy to resources that we tag. So we'll cover tagging and resource groups in general, and there are more things we can do than just automation. But the overriding theme we want to take from this lesson is, hey, we can use tagging and resource groups for automation. So in this lesson, we'll take an overview look at tagging and what resources we can tag and look at some categories and strategies for our tagging. And then just to tie things together, we'll do a quick demo from the management console. We'll see the power of tagging. And then we'll review some exam tips. So with that said, let's take a look at tagging and resource groups. All right, so when we're talking about tagging AWS resources, what can we tag? And it's probably a better question, what can't we tag? There are too many resources to list. But certainly, if you're interested in that, here's the link to view in AWS of all the resources we can tag. But by this time, you've created a lot of resources, certainly EC2 and EBS, RDS databases. Remember, in CloudFormation, there's always the option to add tags. So pretty much anywhere we are within AWS, we have the ability to add tags to resources in AWS. And oftentimes, in these demos, we skip the tagging. And that may be a bit unfortunate because it is important, certainly in an enterprise environment. What if you're dealing with hundreds and even thousands of resources? It then becomes very important because in large environments, the ability to reference a large group of resources, possibly scattered over several regions, and manage them as a group is very efficient and time-saving and certainly eliminates a lot of errors that we may do managing these resources individually. So let's take a look at some of our tagging categories and strategies. So, some of our categories would be technical tags, tags for automations, business tags, and security tags. And looking at our graphic here, we're dealing with key value pairs, and we've dealt with these throughout AWS. It's basically a key and a value. And that's how it works throughout all of tagging. So whatever we give the key, owner, stack, then it has a value. And it's that simple. And we can refer to these key value pairs for technical tags, just identifying some of our resources, business tags, maybe costing, cost allocation, cost reports, security. We can use tags for securing some of our resources. And most importantly for us, tags for automations and we'll see how that works in the demo. Now, strategies. These sort of map to categories, resource organizations, cost allocation, automation, access control, and governance. These two sort of map to security, automation to obviously automation, cost allocation, business tags, resource organization. Again, just identifying the resources we have in our environment. So tagging probably will come up on the exam for you. It's not a huge feature on the exam, a question or two. Just understand what you can do with it. And just about every resource you've created in AWS, you probably had the opportunity to create tags. Going to do a quick demo and show how we can use tagging for automations. And this lesson will actually feed into upcoming lessons on Systems Manager. So this will sort of be the front end of it, and then Systems Manager would execute what we set up in the demo. So let's



see how we can use tagging to facilitate automations in our environment. All right, so I'm in us-east-1 in EC2. I have three instances. They are stopped. And the great point is it doesn't matter. We can add tags to those. And then, over in us-east-2, I have one instance, which is also stopped. So let's go over to Resource groups and Tag editor. So I want to tag those four instances and manage them together. And we need to go into Tag editor. All right, us-east-1. Now recall, we have another instance in us-east-2 for me, Ohio. All right, resource types. We want EC2 instances. And let's search on that. And it shows the four instances, three in us-east-1 and one in us-east-2. Now it's showing that each of these instances have a tag already, and that's fine. Doesn't affect this. We want to add a new tag and create a group for these four instances. So let's select them all, and we want to click Manage tags of selected resources. Now there are existing tags. Let's add one. We'll send our tag key for that new tag. Call it managedinstances, a value, ec2instances. Review and apply changes. Apply changes to all selected. Successfully applied tag changes to all four selected resources. So let's go over to us-east-1, our instances. Click on one of the instances. Go to Tags. Scroll down and there's our tag. Again, it's perfectly fine that it has another tag. Let's trust the other two, and let's just check out the instance in us-east-2. Click on Tags. There's our new tag. So we have a deployment group tag where we can manage these four instances across regions together. Now, I'm going to leave sort of a lesson-ending cliffhanger here. And when we get to a Systems Manager lesson, we will use this tag to manage a deployment to these four instances in our deployment group. So that's sort of part 1 of how we can use tagging to deploy to multiple instances over multiple regions, and stay tuned for part 2 in a Systems Manager lesson. So let's talk about some exam tips. Again, as you create resources, notice most of the resources you created can be tagged, and it's really good practice to tag your resources. But have a strategy for doing it. Don't just do it to do it. Remember our tagging strategies. Resource organization, cost allocation, the big one for us, automation. How do we deploy to multiple instances over multiple region? Well, we can create a tagging group, put all of our instances in that group, and deploy to them based on that tag. Another strategy is access control and governance. This is a lot of security. So that is a look at tagging and resource groups with a focus on how we can use them for our automations. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Amazon QuickSight Dashboards

Hey, Cloud Gurus. Welcome to this lesson on Amazon QuickSight dashboards. So, QuickSight is a service that we may see on the exam. And knowing what it is, what it can do for us, and maybe the services that it interacts with is sufficient knowledge for exam purposes. I think it's sort of a specialized service, and you may or may not encounter it in your DevOps work. But again, it's something that



we should account for and at least know what it is and what it can do for us. So in this lesson, we'll talk about what is QuickSight and what is SPICE? Well, we'll answer that question momentarily. QuickSight is providing dashboards of data. Well, what are the data sources? We'll answer that question. And being that QuickSight is a very visual service, we'll take a look at a demo. Not real in depth, but just to see it and get that visual. And then we'll wrap up with some exam tips. So let's go about answering the question, what is QuickSight? Well, QuickSight lets you create and publish interactive dashboards that include machine learning insights. So behind the scenes, there's some machine learning going on, working with the data that we provide. So QuickSight connects to our data in the cloud and can combine data from many different sources. So in a data dashboard, QuickSight can include AWS data, third-party data, big data, spreadsheet data, SaaS data, B2B data, and more. Now another term you might encounter is SPICE, and that acronym stands for super-fast, parallel, in-memory calculation engine. And when I see that in-memory calculation engine, I think a little bit about Memcached and Redis, some of our other in-memory solutions. Obviously, SPICE is specific to QuickSight. Similar concepts though. In-memory, we're getting that super-fast aspect where we're not traveling too far for that data. SPICE performs advanced calculations and serves data. Storage and processing capacity available on SPICE speeds up the analytical queries that you run against your imported data. So again, that same concept is maybe a Memcached or Redis. We're in-memory, and we're not going far for that data. There you go. SPICE saves time because you don't need to retrieve the data every time you change an analysis or update a visual. I like that analogy. SPICE for QuickSight; Memcached and Redis for our databases. Same concepts. All right. So, where are we getting this data that QuickSight massages and visualizes for us? Let's talk about data sources. QuickSight supports a variety of data sources that you can provide for analysis. Those data sources include Athena, Aurora, OpenSearch, Redshift, S3, RDS, include but not limited to these. Another one might be Tableau. But if you have a handle on this graphic, QuickSight providing that graphic tool for us, massaging data for us, what are our data sources, you'll be in pretty good shape. Possible use cases, analyze log data from services such as AWS Config and CloudWatch. So CloudWatch can send that data to OpenSearch. Data can go to S3. QuickSight can access that data and provide a dashboard for us. So again, if you know what QuickSight is, what it can do for us, and just understand where the data is coming from these data sources, you'll be in good shape. But being that it's such a visual tool, let's just go out to the management console and check out QuickSight. All right, from the management console, let's go to QuickSight. And I just want to show this. We have to sign up for QuickSight, and you get a 30-day free trial. I always set myself alarms for these free trials. Make sure I turn it off. Maybe pause it here and just understand the cost after the free trial. I'll go ahead and click Continue. Different authentications we can use, and I will stick with IAM credentials. Select a region. All right, so we need to enter an account name and email address. There's a QuickSight role, access and auto



discovery. Here are some of our data sources. Now with S3, if you select S3, it'll get more specific as to which bucket you want to use. But I'll leave that unchecked. Maybe just study this list real quick. Just have an understanding of what QuickSight is working with. I'll click Finish, and it's going to create this QuickSight account. All right, just took a couple of minutes to get that set up. Now I can go to QuickSight. Let's just scout around. Dashboards. Well, we can create dashboards. And of course, with a new account, I don't have any dashboards just yet. Create a dashboard by publishing an analysis, ah, which is where our landing page is. All right, let's just go into one of these sample data analysis, website and social media. All right, so it's providing information on page views and visits, visits by device type, subscriber information. Pretty cool. Bunch of others here. Again, I think it's important to understand what QuickSight does for us and the services that it works with. So, I'm not even going to take the time. I don't think it's time well spent to set up a dashboard. Certainly you learn something setting it up, but I don't believe it's necessary for the exam. But there is a page I wanted to show, just like QuickSight related more to DevOps. So here's a dashboard on code change volume, number of code changes made by author and repository. This is cool, meantime to recover dashboard. Outage minutes by application, average time it takes to restore an application. Change failure rate, frequency of deployment failure. So these are really specific to DevOps. Very, very cool. Deployment dashboards. Just go through these a little bit more quickly. Build dashboards. I'll provide this link in the lesson material. That's a look at Amazon QuickSight. All right, let's talk about some exam tips for QuickSight. What is it? With QuickSight, we can create dashboards, interactive dashboards, that include machine learning insights. How can QuickSight be used in a DevOps setting? Well, we attach it to those data sources that we saw such as S3 and RDS, Athena, Aurora, even Tableau. Typical automations can populate these data sources, for example a Lambda function sending CloudWatch Logs to an S3 bucket. So once again, if you just understand what QuickSight does, the service it provides, and the data sources it interacts with, you'll be in good shape. And with that, I will end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

CloudWatch Synthetics

Hey, Cloud Gurus. Welcome to this lesson, AWS CloudWatch Synthetics. So this is a short lesson on Synthetics. Probably not heavily covered on the exam, but certainly something you want to be aware of and just know what it does and how it does it. And if you know the use case for Synthetics, it will serve you well on the exam. All right, so what are CloudWatch Synthetics? It's an automated approach to monitoring the performance and availability of your application endpoints, REST APIs, and website content, allowing you to discover issues before your customers do. So with Synthetics, you can continually verify your customer's experience, even when you don't have any traffic on your applications. So in a sense, it mimics what your customers are doing and searches



for any issues. Of course, CloudWatch Synthetics, it's a part of CloudWatch. The testing can be accomplished using canaries. So we can test our application endpoints, web apps, APIs, all with these canaries. So what are canaries? Well, more on canaries in the next slide, but just think about canary testing in DevOps. So in other areas of AWS such as API Gateway, we might do a canary deployment. So we're reducing the risk, directing a smaller percentage of our traffic to this canary deployment. Now, it's not quite the same as what CloudWatch canaries are doing, but the same outcome can be determined. Hey, is this working okay? And in the case of CloudWatch Synthetics, these canaries test our application endpoints, our web app and our APIs. Is everything still working okay? And again, the main purpose is to find out problems before our customers do. If we find the problem and fix it before anybody realizes there's a problem well, was there a problem? All right, a little bit more on canaries. They are actually scripts. Canaries are configurable CloudWatch synthetic scripts used to monitor web applications. So we can set these scripts up to periodically monitor our endpoints and, again, sort of mimic the end user experience. So in a little bit more detail, and this is key, again, just for your knowledge of Synthetics and its interactions. Canary scripts create Lambda functions. The Lambda functions expose an interface through which you can inject code, example being Node.js. This code performs endpoint configuration, authentication, and validation of responses. That's the whole ball of wax, so to speak. Is our app up? Is it sending back appropriate and timely responses? We can determine all of that with CloudWatch Synthetics and these canary scripts and ideally find problems before our customers run into those problems. And as you might expect, you can log results and generate alarms for values that fall beyond preconfigured thresholds. So the canaries are interacting with Lambda and, of course, CloudWatch and specifically CloudWatch Alarms. So we're getting log files. Well, we can store those log files in S3 and CloudWatch logs, another storage area. And we can set alarms as stated before based on our preconfigured thresholds. So that's a look at CloudWatch Synthetics and these canary scripts. And as always, just understand the other services that our Synthetics and our canaries can interact with, most importantly Lambda, S3 buckets, CloudWatch Logs, and CloudWatch Alarms. And with that, I'll end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Amazon Lookout for Metrics

Hey, Cloud Gurus. Welcome to this lesson, Amazon Lookout for Metrics. And Lookout for Metrics is very closely related to our last lesson, CloudWatch Anomaly Detection. Lookout for Metrics automatically detects anomalies within metrics and identifies their root cause. So the root cause analysis takes things a step further. Additionally, Lookout for Metrics can actually sit between some of your services where you want to detect anomalies in CloudWatch. So it's really not in either/or. You can certainly use both of these for detecting anomalies



in your data. So let's check out Lookout for Metrics. All right, what is Amazon Lookout for Metrics? It's a service that finds anomalies in your data, determines their root causes, very key point, and enables you to quickly take action. So with the CloudWatch anomaly detector, we saw where we can set alarms. Again, Lookout for Metrics, that key point determine root causes. Reduce false positives and use machine learning to accurately detect anomalies in business metrics. Certainly, that's very similar to the CloudWatch anomaly detector. Here's where we differ. Diagnose the root cause of anomalies by grouping related outliers together. Causes and rank them by severity. I always like to use analogies. AWS Security Hub can find security issues and rank them by severity. We can integrate databases, storage services, and third-party SaaS applications to monitor metrics and detect anomalies. And we can automate customized alerts and actions when anomalies are detected. And yes, Lookout for Metrics can interact with CloudWatch. All right, continuing further. What is Lookout for Metrics? So we have data sources that can be sending data to Lookout for Metrics. And I'm showing S3, Redshift, CloudWatch. We'll see momentarily in a brief demo all of the services we can set up to work with Lookout. So we connect our data to Lookout for Metrics. Lookout for Metrics will use machine learning to detect abnormal changes in our metrics. And then again, it's going to create that impact summary, and it's going to rank those abnormalities in terms of severity. So again, certainly the machine learning to detect abnormal changes, anomalies. Very similar to CloudWatch anomaly detection. Where Lookout kicks it up a notch, so to speak, create an impact summary to understand severity and root causes. And as with most AWS services, we can create that notification chain, automated alerts and custom actions. SNS or maybe a Lambda function to take a corrective action. All right, let's look at a demo. Just go out to the management console. Always like to study those drop-downs. It sort of paints the picture. I think of what services can interact with Lookout. Key point here, when an anomaly detector processes data, Lookout for Metrics sends metrics to Amazon CloudWatch. I want to emphasize that point because could be on the input or the output side of this. We'll see very soon in the demo, yes, we can get data from CloudWatch if we choose to and also Lookout and send data to CloudWatch. And we can build graphs and dashboards from this data, again, from several different services. So let's just see how we set up Lookout for Metrics. All right, I'm out at the management console. Lookout is here in my recently visited, but you can certainly just do a search on Lookout. This How it works is informative. We create a detector. That's what we're going to do. Select the data we want to monitor. Activate the detector, add an alert, send alerts to Lambda, Webhooks, maybe Slack, PagerDuty, Datadog, SNS, and, of course, view the anomalies. Let's just create a detector. We did a little bit of doing anomalies on a graph in the last lesson. So we'll just create a detector. Set an interval, the amount of time, let's say an hour. And let's create, and it just shows these helpful steps again. Let's take this a step further. Add a dataset. Choose a time zone. And one thing I really want to show here, choose the data source that stores or generates your data. This drop-down is key. Well,



we have some options here, S3, CloudWatch, a lot of data storage, Redshift, Athena, Aurora, MarieDB, SQL, MySQL, PostgreSQL, Zendesk. I'll just choose S3. So do we want to find anomalies in historical data all at once, or do we just want to start from right now? I'll just choose Continuous. Lookout for Metrics will attempt to automatically detect the pattern of the S3 path to each object in your continuous dataset. Let me back out of this. I'll just show another dataset from a different source. Let's see what CloudWatch looks like. To run the detector and backtest mode, use Amazon S3 or CloudWatch for the data source. Detect anomalies and data from other services that integrate with CloudWatch. So this is the data being sent to CloudWatch. I'll back out of this. Let me just show one more. Let's see what MySQL looks like. A lot of what we might expect we need to provide when working with a database. DB identifier, point a host DNS name, port number, database name, table name, and some VPC and security group information for our database instance. All right, I just wanted to get the flavor of how this works. And between the CloudWatch anomaly detector and Amazon Lookout for Metrics, we certainly have options for anomaly detection using machine learning. And for the exam, just know each of these services and what they do basically. And just remember, Lookout for Metrics is going to provide a severity report and a little bit of a root cause analysis for us. So with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

AWS Distro for OpenTelemetry

Hey, Clock Gurus. Welcome to this lesson, AWS Distro for OpenTelemetry. So in this lesson, we'll see what AWS Distro for OpenTelemetry is, what it can do for us, and its close connection with another AWS service. Think about that one. Think about the word telemetry. But we'll find out the mystery shortly. All right, what is AWS Distro for OpenTelemetry, acronym ADOT? OpenTelemetry provides a single set of open-source APIs, libraries, and agents to collect distributed traces and metrics. And there's another hint on the related AWS service. And it is for application monitoring, yet another hint. Can speed up performance, troubleshooting, and reduce meantime to resolution with increased visibility into your AWS resources. We can instrument our application just once, then send metrics and traces to multiple AWS monitoring solutions. All right, so first of all, let's dissect that name, AWS Distro for OpenTelemetry. OpenTelemetry is a project, an open-source project. AWS has created a distribution of that project. Let's go a little further with this. All right, so it collects traces, can collect application metrics, traces and metrics, contextual data. Ah, there's our similar service, AWS X Ray. We can use one or the other, or they can work together. So Distro for OpenTelemetry can send these traces and metrics to X Ray, CloudWatch, Managed Service for Prometheus. A key point of all this is, okay. X Ray is AWS-specific, and AWS Distro for OpenTelemetry is a distribution of this open-source telemetry. So, if we want to send this data outside of AWS to



some partner solutions, this is the answer, AWS Distro for OpenTelemetry. If we're using X Ray, we can migrate our X Ray data to Distro for OpenTelemetry and then pass it outside of AWS. Let's just look at this from a different perspective. Some of our inputs to OpenTelemetry could include X Ray, OpenSearch, Kafka, and we can migrate X Ray data to OpenTelemetry. Now the outputs, somewhat predictable, Prometheus, CloudWatch. Yes, we can send to X Ray as well. Key key point, we can send to partner monitoring solutions. So, the situation could be we're using X Ray. We migrate to Distro for OpenTelemetry and ultimately send that to partner monitoring solutions outside of AWS based on that standardized set of data. So you may see a mention of AWS Distro for OpenTelemetry or OpenTelemetry. Now you know what it is, very closely related to X Ray and can be a liaison between AWS and the outside world. With that, I will end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

CloudWatch Anomaly Detection

Hey, Cloud Gurus. Welcome to this lesson, AWS CloudWatch Anomaly Detection. CloudWatch anomaly detection applies statistical and machine learning algorithms to CloudWatch Metrics. It calculates normal baselines and minimizes surface with minimal user intervention. So using CPU utilization for EC2 as an example. Rather than us giving a best estimate of upper and lower percentages for CPU utilization and our CloudWatch alarms, anomaly detection will provide statistics to determine these lower and upper boundaries. All right, why do we need an anomaly detection? All right, let's look at a common problem. We're using an alarm based on CPU utilization, but we keep getting false alarms. So we're doing this throughout our enterprise and have many different use cases, but this same problem exists. We keep getting these false alarms. So when we set up EC2, we're used to setting these alarms, setting the upper and lower limits on CPU utilization. And we have a notification chain. If an alarm is triggered, we send notifications. But guess what? The accuracy of this is not great. And a result of this inaccuracy is that we could get scaling when it's truly not necessary. All right, so what are other potential problems? Well, the static alarms don't support a dynamic range threshold. We get frequent false alarms, and it's a maintenance-heavy solution. As you can imagine, if we're constantly getting false alarms, we're going to be troubleshooting this. An added complication is, well, what if we're doing this all over our organization? And it's difficult to calibrate across many different use cases. Additionally, while we're doing it this way, we aren't learning historical trends, which would make our metrics and subsequently our alarms more accurate. All right, we have a pretty good idea by this point. But what is CloudWatch anomaly detection? It applies statistical and machine learning algorithms to CloudWatch Metrics. It then calculates normal baselines and minimizes surface anomalies with minimal user intervention. So the key takeaways from that is well, hey, it calculates rather than us guessing normal baselines and minimal user intervention. Once we set it up,



it's working for us constantly. And to sort of summarize that, it's using our historical data to learn on the job and give us better baselines. So rather than guessing our upper and lower limits, we put anomaly detection in place. The algorithms generate an anomaly detection model. The model generates a range of expected values that represent normal metric behavior. It's learning from our historical data, and it's more accurate. So we can use the model of expected values in two ways. Create anomaly detection alarms based on a metric's expected value. And certainly, if we don't have a lot of experience working with this metric, this is a better option for us. And to further refine things, when viewing a graph of metric data, we can overlay the expected values onto the graph and analyze what we're seeing visually on the graph. So let's see how this works with a demo. We'll apply machine learning algorithms to the metric's passed data. That's where our CloudWatch anomaly detection comes into play. And remember, it's working on the historical data of our metric. And we're just going to stick with that CPU utilization metric. From that historical data, it's going to determine expected values. And it's going to create a model of our CPU utilization's expected values. The model assesses both trends and hourly, daily, and weekly patterns of the metric. All right, so let's nail down these concepts. With our demo out at the management console and also a bit from command line interface, and let's set up CloudWatch anomaly detection. All right, out at the management console. I have two tabs open in EC2 and also CloudWatch, and I have one instance running. We're going to work with that instance. Now let's go over to CloudWatch, and let's go into Metrics. Let's click on EC2, Per-Instance Metrics. We can sort those by name. Actually did that twice, starting at the beginning of the alphabet. We want CPU utilization. Here is our instance. We have it tagged, so the instance ID isn't as important. That's our metric. Let's click Graphed metrics, and here is a previous anomaly detection that I ran. But let's stick with this CPU utilization. What I can do is click on this alarm. Now if we scroll down a little bit, conditions right now of static. We want anomaly detection. And watch this change. All right, what's an anomaly? When do we detect it? Outside of the band? Greater than, lower than? So, basically, this is both. Let's stick with that. Let's click Next. Notice, alarm state trigger. We can set up an SNS topic. Now we've done these notification chains many times. I'm not going to get too deep into this. The alarm's going to be our trigger, and we'll send a message. I've already created an SNS topic, CWAnomaly. And I just have to subscribe to that. Ultimately, the alarm will trigger SNS. I get an email. I'll just scroll slowly. Notice some other things we can do here. But I'm just going to stick with SNS. Click Next. Let's give our alarm a name and click Next. Oh, here we have a typical AWS review page. Just review everything we set up, and we can create our alarm. As expected, our alarm has insufficient data right now. Now let's go back over to EC2. Here's our instance. I'm going to connect to this instance. I'll use Instance Connect. Click Connect. Let's test a bit. And what I'm going to do is stress this server with a stress tool, and we need to install that. All right, command is `amazon-linux-extras install epel -y`. Intentionally want to show something here. You lack permissions to write to system configuration. How do we solve



that? We use the sudo command, and epel is being installed. Now, typically, this will take 3 to 5 minutes. I already have epel on this machine, so there wasn't really anything to do. So our stress tool is within the package we just installed. Let's install the stress tool. Let's see what happens. I already have that on here. But I did want to show the command. All right, nothing to do. But naturally, if you don't have it on your machine, it will be installed. So, we have stress installed. Now we can run it. So it's not going to run forever. It'll time out. I'll run this. So it's dispatching hogs to hog up our CPU. Now, I Ctrl+C-ed out of that, but it's still running in the background. And just to verify that, I'll run the top command. And we have stress running eight times. Remember that CPU number of eight. So we are definitely stressing our server. Let's go back to CloudWatch. Ah, our alarm is in alarm, as it should be. Now this might take a little while. But ultimately, that stress we're putting on our server will trigger that alarm. Now what else is going on and what happened? Anomaly detection is collecting data to establish a baseline. So let's click on our alarm, and that's pretty obvious how our CPU utilization just skyrocketed. Notice that in red, 5.31, greater than expected. Well, our CPU utilization just pegs, 88.1% greater than expected. So it's learning from this experience. Now we're not going to hang around. But remember, that stress tool is going to run for 800 seconds. Now the important part of this is, okay, after 800 seconds, our stress tool times out, our CPU utilization drops. But whatever's going on, high stress, no stress, 30% CPU utilization, anomaly detection is analyzing this metric, learning from it, and setting those boundaries based on that historical knowledge. And with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

Logging

Encrypting Logs

Hey, Cloud Gurus. Welcome to this lesson, Encrypting Logs. So we've talked a good bit about CloudWatch Logs, and it's just appropriate at this point to talk about how we secure these logs. And of course, encryption is the answer to that. We'll talk about data protection in general, and then we'll talk about Key Management Service a bit. We'll talk about encrypting CloudTrail logs. And we'll wrap up this quick lesson with some exam tips. So let's get started and talk about data protection. So, let's look at some general ways to secure our data. Well, right off the bat, we can use multi-factor authentication with accounts. We can use SSL and TLS to communicate with our AWS resources. And we can keep an eye on our API activity and user activity logging with CloudTrail. So that's all on top of before we even get into encryption, or at least encryption of data at rest. Let's talk a little bit about log encryption. So it's important to understand that CloudWatch Logs are automatically encrypted at rest. Log group data is always encrypted in CloudWatch Logs. CloudWatch Logs manages server-side



encryption keys. So, what are we talking about here? If it's already done for us, well, maybe there are compliance reasons where we need to manage this ourselves. So what if we need to manage those keys? That's where Key Management Service KMS comes into play. And we can manage the keys using a customer master key from KMS. So let's talk about KMS a little bit. So, we associate a key with a log group, either when we create the log or we can even create the key after it exists. So all newly ingested data for the log group is encrypted using this key. Key point there, all newly ingested data. CloudWatch Logs decrypt this data whenever it is requested. And you can just think about that if you've ever looked at CloudWatch Logs, well, we just have to request them and then we can read them. CloudWatch Logs must have permissions for the customer-managed key whenever encrypted data is requested. What about CloudTrail? What about those logs for APIs? So, when CloudTrail sends log files to S3, it's using this encryption, SSE-S3. Again, it's already encrypted for us by default using Amazon S3-managed encryption keys. SSE-S3, I'd say just make sure for the exam you understand what these acronyms are. You might see these SSE-S3, SSE-KMS. And as you might expect, this is the AWS-managed version. What if we need to— Maybe, again, it's a compliance issue where we need to manage these keys. We would use SSE-KMS. So we create a KMS key. We create a policy and attach it to the key. And then the key determines which users can use the key for encrypting and decrypting CloudTrail log files. So, think about with CloudTrail log files maybe for audits and/or compliance, we need to make sure those log files aren't tampered with. That might be a situation for SSE-KMS. All right, just wanted to cover encryption on our log files quickly. With all that said, let's check out a couple exam tips. Understand the different types of encryption that can be used with log files. Again, by default, CloudWatch Logs manages server-side encryption keys, and we also saw that with CloudTrail. Are there other options? Yes, there are. KMS using a customer master key, and we can control that if we need to. What about CloudTrail logs? Same deal. They're automatically encrypted in our S3 bucket using SSE-S3. If we need to manage that, we can do that with SSE-KMS, and I just answered this question. All right, that is a look at encrypting our logs, CloudWatch and CloudTrail. So I will end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

API Gateway Monitoring and Logging

Hey, Cloud Gurus. Welcome to this lesson, AWS API Gateway Monitoring and Logging. So in this lesson, we'll see how we can set up monitoring and logging for API Gateway, and we're going to see that a lot of the usual suspects are involved here. So let's talk about our monitoring and logging options with API Gateway. Well, as I'm sure you would expect, CloudWatch Logs are available to work with API Gateway. And with that, we can log API calls, and it's a helpful tool for debugging. CloudWatch alarms can also be used with API Gateway. And much like we would use with, say, EC2, we can set an alarm on a single metric to set up



notification chains. Kinesis Firehose is another option. To help debug issues related to client access to your API, you can enable Firehose to log API calls. So just remember, CloudWatch Logs is not the only option for logging API calls. CloudTrail and, of course, API calls are involved here, but not so much troubleshooting as maybe auditing. Who did what from where. CloudTrail can provide a record of actions taken by a user, role, or an AWS service in API Gateway. X-Ray may be the surprise guest in this group. But we know what X-Ray does, identify issues with your application and opportunities for optimization. So, the most common scenario where we see API Gateway is as a front end, maybe to a Lambda function, in a database. And X-Ray can trace the whole way through front to back, identify issues, and suggest optimizations. And we can certainly monitor API Gateway with Config. Config rules represent the ideal configuration settings for the API Gateway resources. And of course, if something strays from that ideal configuration, Config will record that, or we could set up a notification chain. Let's focus on CloudWatch, setting up CloudWatch API logging in API Gateway. All right, so we have a dev and we have a prod API, and CloudWatch Logs comes into the picture. Here's the key point. To set up CloudWatch API logging, you must have deployed the API to a stage. So that's fine that we have a dev and we have a prod API, but we need to deploy those first. And as is generally the case, we have a CloudWatch log role, which will have a policy granting permissions to work with our APIs. All right, let's wrap up with the demo and see how we can set up CloudWatch API logging. All right, before we get started in the management console, let's just take a look at that policy. And this policy will grant API Gateway permission to read and write logs to CloudWatch. And actually, this is an AWS-managed policy named `AmazonAPIGatewayPushToCloudWatchLogs`. So we would create our IAM role and attach this policy to it, and then we'll be ready to set up CloudWatch API logging. So let's go to the management console. Let's go to Settings and choose Edit. And we need an ARN for the role. All right, so I'm going to bounce over to IAM and search for my role. I created this role. Showed creating roles several times, so I didn't want to show that again. There's our policy. Here's our ARN that we need. Copy that, plug that in, save changes. All right, now let's go over to our APIs. And I'm going to work with, again, this Lambda Proxy API. Click on Stages. All right, and this API only has the one stage right down here, logs and tracing. Click Edit. Ah, CloudWatch Logs off. So, I'll open the drop-down, and we'll see that we have logging levels. Of course, it's off right now. Errors only, errors and info logs, full request and response logs. And full request and response logs is not recommended for production APIs. And I'll just select Errors only. We could turn on detailed metrics. Each method will generate these metrics, API calls, latency integration, latency, 400 and 500 errors. We could turn on custom access logging. And notice, CloudWatch log group or Firehose stream, and we can turn on X-Ray tracing. I'll just leave it as is and save the changes. And to wrap up this lesson, let's bounce over to CloudWatch. And let's go to Logs, Log groups, `aws/api/gateway`, created this log group 6 minutes ago. This is our log group for API Gateway logging. And I don't expect any errors at this point, but this is where we



would go to start investigating. And with that, I'll end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.

ECS Logging

Hey, Cloud Gurus. Welcome to this lesson, ECS Logging. And just like with EC2, we can perform logging and monitoring on our ECS containers. So I'll go over what's available and do an abbreviated demo on how to configure logging for ECS. So let's check out ECS logging. And of course, to do some logging, we're probably going to be monitoring some activities, and there are several tools provided for monitoring your ECS resources and responding to potential incidents. So we have CloudWatch Alarms. And with Alarms, we can monitor a single metric over a time period and perform actions based on the value of that metric. So we've seen this in action certainly with EC2, and we could set up notification chains with SNS. And yes, if you were wondering, Trusted Advisor can monitor ECS for us. It can inspect our containers and make recommendations for cost savings, system availability, and performance for security issues. Of course, we have logs, CloudWatch logging. And momentarily, we'll talk a little bit more in depth about CloudWatch Logs and how to set that up for ECS. We can use EventBridge events with ECS and route those events to targets or streams to make changes or capture state information or take corrective action. Also, CloudTrail certainly does its thing with ECS, and that thing is CloudTrail is going to provide a record of actions taken by a user, role, or an AWS service in ECS. We can determine the request that was made, the IP address from which the request was made, who made the request, when it was made, and a few additional details. And somewhat new to the game, Compute Optimizer analyzes our configuration and utilization metrics. And it can report when our resources are optimal and also generate optimization recommendations that can reduce cost and improve performance. And the segue into the next slide, we can configure the containers in our task to send log information to CloudWatch Logs. So let's talk about that. All right, so we have our ECS cluster and containers within that cluster. And the key task is to monitor this cluster and send log application data to CloudWatch Logs. So how do we do that? And this is not an in-depth demo, but we'll certainly get the idea and go in detail on some commands. First thing we need to do is create a log group, and we do that with the command `aws logs create-log-group`. And then all we need to do is add a new log configuration, which will contain the name of that log group we just created to the container task definition. Now let's just wrap this up by taking a look at a couple code snippets. And the disclaimer here, I'm just using Visual Studio Code to show some code snippets. This is a CLI command. It certainly doesn't belong in a JSON file. Just here for convenience. So, to create our log group, we would create a little file, `ecs-log-group.json`. Log group name, `ecs/`, and whatever we want to name it. And then just set the retention days. So once we have that file, we run our `create-log-group` command against this `log-group.json`



file. So that's going to create our log group. Now as far as the task definition, we just have to add couple of lines to our task definition, logConfiguration, logDriver, awslogs. Here's the important part, aws logs-group and the name of our container. Here, it's specifying the region. And logs-stream-prefix, of course, is ECS. So, really with a couple of lines of code between the task definition and this little JSON file, then we execute the command, we will have set up logging from our ECS containers to CloudWatch Logs. So with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

EKS Logging

Welcome to this lesson, AWS EKS Logging. And in this lesson, we'll talk about a few of our options for logging. And with EKS, we have a couple different options for logging and getting our logs to CloudWatch. So let's get an overview look at EKS logging. From a high level, EKS can integrate with CloudWatch Logs for the Kubernetes control plane. And we do have one key factor here. With the control plane, we can turn on logging without installing the CloudWatch agent. So we can send in logs to CloudWatch. We don't even need the CloudWatch agent. And more specifically, those logs will be control plane, audit, and diagnostic logs. Additionally, CloudWatch Container Insights provides a comprehensive metrics monitoring solution for EKS at the cluster, node, pod, task, and service level. So again, these are control plane logs. Control plane can send those to CloudWatch without the CloudWatch agent. Now we have more logs and a different way to get those to CloudWatch. Let's talk a little bit more about the control plane just to emphasize that point. Control plane nodes are integrated with CloudWatch, and you can turn on logging for specific control plane components. The important part about that is, yes, you can send all of those control plane logs. What if you don't want all of them? Well, you can be selective about that. Now we mentioned Container Insights. Container Insights implements cluster node and pod-level metrics with the CloudWatch agent. So, here's the difference. We have control plane logging, but we also have logs for our clusters and our nodes and pod-level metrics. With those, note, we do need the CloudWatch agent and Fluent Bit or Fluentd for log capture to CloudWatch. So we have control plane logging and sort of everything else, node logging, application logging. We need the CloudWatch agent, and we need this Fluentd. Fluentd is an open-source data collector for a unified logging layer. And then there's Fluent Bit. They go hand in hand. Fluent Bit, super fast, lightweight, highly scalable logging and metrics processor and forwarder. So really bottom line, control plane logging, we don't need the CloudWatch agent. Control plane can send those logs right to CloudWatch. Clusters, nodes, yes, we need the CloudWatch agent, and we also use Fluentd and Fluent Bit to get those logs to CloudWatch. And that's probably more than enough knowledge on EKS logging. And we'll be set for the exam if we run into anything regarding EKS logging. So with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I'll



see you in the next lesson.

Monitoring and Logging Review

Supporting Services for Monitoring and Logging

Welcome to this lesson, Supporting Services for Monitoring and Logging. So this is another lesson where we'll focus on the monitoring and logging domain and look at some services that they might not be real prominent on the exam, but you may run into them. So it's best to know what they are and what they do and how they fit into the DevOps picture. So we'll take a look at AWS Compute Optimizer, AWS License Manager, and the AWS versions of Prometheus and Grafana, Amazon AppFlow. And then we'll finish off with some exam tips. So let's go ahead and check out Compute Optimizer. So Compute Optimizer is going to examine our workloads, and it's going to provide optimization recommendations. It uses machine learning to analyze how our resources are configured, and it's going to help us avoid overprovisioning and underprovisioning. So if we're overprovisioned, we're wasting money. If we're underprovisioned, our performance is suffering. And Compute Optimizer can analyze four types of resources, as you probably expect, EC2, EBS volumes, ECS services on Fargate, and Lambda Functions. And of course, you have the opportunity to right-size these when you create these. But we're not always perfect, and Optimizer can help us. And as is usually the case, often the case, CloudWatch is in the mix. Compute Optimizer can analyze resource utilization data from CloudWatch. So remember, with CloudWatch, we have that utilization data such as CPU utilization. We can set alarms on that. Well, Optimizer can get at that data as well. So that's Compute Optimizer in a nutshell. Just remember the four services it works with, and it provides optimization recommendations so we don't over or underprovision. Let's check out License Manager. And that name is very descriptive. It manages your software licenses from software vendors such as Microsoft, SAP, Oracle, and IBM. Manages licenses across your AWS and on-premise environments. So we have License Manager, and we can define rules for licensed software. We can attach those rules during launch of our software. Can also search inventory and track licenses brought in post launch. So it's a continuing process, not just that launch. And we can use alerts to control and centrally manage licenses across all AWS accounts. And just remember, on-premises as well. So that can reach into on-premise. All right, let's check out AWS Managed Service for Prometheus, otherwise known as AMP, and Amazon Managed Grafana. So, Prometheus monitors and provides alerts on containerized applications and infrastructure at scale. So, AMP can be integrated with Elastic Kubernetes Service, Elastic Container Service, and AWS Distro. So the connection to make is Prometheus to containerized applications. And just in case you don't know, so Amazon Managed Service for Prometheus suggests that



there's probably an open source version of Prometheus out there, and there is. Prometheus is a software application used for event monitoring and alerting. It records real-time metrics in a time series database using an HTTP pool model. So that is AMP, and let's take a look at how it is implemented and also bring Grafana into play. Let's define Grafana, a fully managed service that enables you to query, visualize, and alert on your metrics, logs, and traces. It works hand in hand with Prometheus and with Grafana. We can get charts and graphs and alerts. All right, but let's see how this breaks down. So we have Elastic Kubernetes Service running in our VPC. And what we do is create an AMP workspace, and we run a Prometheus server to export the application metrics over here on Kubernetes to AMP. So we're getting all that metric data into AWS Managed Prometheus. And we can configure a Grafana server to query this AMP workspace, get all that metric data, and display it graphically with charts and graphs. I think the big takeaway for these services is they work together, Grafana can chart and graph the data, and AMP works with containerized applications. All right, let's check out Amazon AppFlow, automates bidirectional data flows between SaaS applications and AWS services. All right, so we have some SaaS applications, Salesforce, SAP, Google Analytics. So what does AppFlow do for us? Well, it's going to ingest that data from SaaS applications. It can enrich that data through filters and validations. And as you might expect, it's going to transfer that data to our data stores. Could put it in S3. And if we want to do some deeper analysis, put it in Redshift. Key takeaways, AppFlow is a managed service. So it's going to scale for us automatically. And again, just remember, it works with SAS applications and just have a general idea of what they may be. All right, those are some supporting services for monitoring and logging. Let's look at some exam tips. What services can Compute Optimizer work with? Remember, there are four, EC2, EBS volumes, ECS on Fargate, and Lambda Functions. And Compute Optimizer can tell us if we're over or underprovisioning, either save us some money or better our performance. Where can AWS License Manager be used? Just a key point. Certainly in AWS accounts, but also we can manage our on-premises licenses as well. All right. What acronym is commonly used to refer to Amazon Managed Service for Prometheus? And just be ready for this if you see the acronym, AMP. And with that, I will end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in the next lesson.

DevOps Match Game for Monitoring and Logging

Hey, Cloud Gurus. Welcome to this lesson, DevOps Match Game for Monitoring and Logging. And with this lesson, just want to gamify the learning process a little bit and really train our minds to know the fundamentals of services and use those fundamentals to help us get through scenario questions quicker, specifically by being able to eliminate incorrect answers. If we see a key term or a phrase that points to a certain solution, well, we start scanning those answers looking for that specific service. If it's not there, we don't have to read further on that answer. We



eliminate it. Can really speed up the process, save us time and brain power, and help us finish the exam strong. So in this lesson, we'll have a match game on data protection and compare CloudWatch with CloudTrail and AWS Config. So let's go ahead and check out the match game for data protection. All right, so the thought process here is we have a scenario question with multiple sentences. And these items on the left are a sentence in that scenario question, and they really provide clues that can speed up our process. So let's see how this works. So, provides API event and user activity logging. So, if I see API, I'm immediately thinking CloudTrail, and then I start scanning those answers. And if it doesn't have CloudTrail in it, I'm not wasting my time or my mental energy reading the answer. I go on, eliminate that one, and then I have a smaller answer set. All right, next one. Can discover and secure sensitive data in S3. Well, there's only one service that can do that. If I see this, I'm looking for Macie in my answer set. Eliminate the ones that don't have it. Next one, safeguards data in transit between AWS resources. In transit, there's our key term, SSL/TLS. All right, is an IAM best practice. Well, there are a lot of IAM best practices, but I only see one IAM best practice on the right, and that is multi-factor authentication. Now, relating this to services in AWS, Security Hub will produce a finding if you do not have multi-factor authentication on the root account. All right, finally, an alternative way of encrypting CloudWatch data at rest, Key Management Service. So, CloudWatch data is encrypted at rest automatically. But if you want to manage those encryption keys, maybe it's a compliance requirement, you can use Key Management Service. All right, let's check out another match game. CloudWatch versus CloudTrail versus AWS Config, and we just have a note here. CloudWatch agent can be installed on EC2 and on-premises instances. All right, remember, the premise. We have scenario questions with multiple sentences, and these ones on the left are a sentence in that scenario. Records when and where a change was made. When and where, that's a big clue. The other clue for this service, this is a hint. If you see API, think this service as well, CloudTrail. Key terms for CloudTrail, API records when and where a change was made. If you see that, your answer better have CloudTrail in it. Tracks which AWS resources have changed. It's interesting when you think about it and just go through that thought process real quick. That was fairly easy, I think, I hope. Collects log files and data, and log files is my key term there. I'm thinking CloudWatch Logs. There we go. Records who or what made a change. This sounds familiar to the top one. It's another key feature of CloudTrail. If you see that, start thinking about your answer including CloudTrail. Collects and tracks metrics. Hopefully this one's fairly easy, CloudWatch, CloudWatch Metrics. Kind of gave this one away already. That is really a key word for CloudTrail, API. And finally, provides detailed snapshots regarding how resources were changed. AWS Config. So, AWS Config can tell us how resources were changed. CloudTrail gives us when and where a change was made, who made that change. So if you really know these fundamentals between CloudWatch, CloudTrail, and AWS Config, certainly score some points on the exam and save yourself some time. And with that, I will end this lesson. Thank you for joining me, Cloud Gurus. I'll see you in



the next lesson.

Monitoring and Logging Scenarios

Hey, Cloud Gurus. Welcome. So in this lesson, Monitoring and Logging Scenarios, this is our end-of-section wrap-up. Some of what we'll cover in here will be review, and there may be a new thing or two. But if it's not covered in the course, most likely covered in the associated whitepapers. And one thing I like to do before sitting for the exam is for some of the main services is scan the FAQs real quick and make sure I recognize everything I'm seeing in the FAQs so I don't get any surprises on the exam. So with that said, we're just going to sort of describe the exercise. We've done one of these already. And then we're just going to talk about three scenarios. So let's go ahead and talk about what we're trying to accomplish in this exercise. All right, so the exam places some challenges on us, and the main challenge is reading through the scenario, sort of visualizing the architecture described in the scenario. So what we're going to do here is practice that a little bit. Now the first thing we want to do is visualize that scenario. And then as part of the exercise, sketch the scenario on a notepad, like a notepad were provided in the professional exam. And then finally, we'll click through the diagram to reveal the solution and compare it to what you came up with. Now remember, on the real exam, we have roughly 2 minutes per question and we're not going to have time to sketch out every scenario. We sort of have to pick and choose. And choose and the ones we may be struggling with, maybe sketch that out. What I like to do on the exam, if I'm struggling with a question— Well, first of all, I don't struggle with a question the first time around. If it looks difficult to me, I mark it for review and move on. I like to move fast through the exam and sort of pick off the low-hanging fruit. Get some questions under my belt, get some confidence going. And amazingly, when I mark the hard questions for review, when I see that question the second time around, it almost always seems a little bit easier. And step 2, yes, we want to sketch it out. But the most important thing is try to visualize these scenarios. So let's check out some scenarios for monitoring and logging. All right, we have a hybrid environment that includes several on-premises servers. What can we do to collect the application logs from these servers? Now this should be something that we certainly covered. So whenever you want, just go ahead and pause the video and visualize the scenario and maybe sketch it out. All right, so we have our on-premises servers. Now the key to this is to realize that, okay, this isn't an EC2 instance, but we can still do AWS things to these servers. And what we can do is install the CloudWatch Logs agent. Remember, any time we want to get those CloudWatch Logs, we install the CloudWatch Logs agent, whether it's on an EC2 instance or an on-premises server. All right, let's check out scenario 2, and it sort of builds on this. We get a new requirement to run real-time analysis on the application logs. What steps can we take? And again, pause the lesson, visualize your scenario, and sketch out the scenario. But let's see what we have here. All right, so we have our on-



premises server. We have the CloudWatch Logs agent. Now the key to this little scenario, here's our magic term, real-time analysis. What's real time usually screaming to us? It is screaming Kinesis. There's our Kinesis icon. Now, this solution is a little bit different, and I don't have time to cover every little detail of AWS in the lesson, but covering it now. In CloudWatch Logs, what we can do is create a subscription filter. Now we can do that with Kinesis, or we can do that with Lambda. So we create a Kinesis subscription filter, and we're streaming this data to Kinesis for analysis. Intentionally not overcomplicating these scenarios. But obviously, on the back end here, we'd be doing some analysis. But the main point in this scenario is these Kinesis subscription filters. And remember, we can do a Lambda subscription filter as well. And just reemphasizing that linkage real time to Kinesis. And certainly, Lambda we can do real time sort of event-driven architectures as well. All right, let's take a look at another scenario. We have multiple AWS accounts and need to consolidate the CloudTrail log files in one location. All right, first of all, where would we consolidate these files? So picture that in your mind. Additionally, we have an audit coming up and need to ensure the log files are not altered. And we did cover that in the course. Go ahead and pause and visualize and make your sketch. All right, multiple accounts. So each of these accounts, obviously, we can set up a CloudTrail trail. Need to consolidate in one location. Where would we do that? We would do that in S3. So all of our accounts can send CloudTrail logs to one S3 bucket in the region. What else do we need to do? We configure cross-account access on the bucket, set the bucket as the destination for CloudTrail on all of the accounts. Now how do we handle this need to ensure the log files are not altered? We enable log file validation on CloudTrail. And in an audit scenario, it would be important to be able to prove that your log files have not been altered. Questions on the exam can be difficult, but the key is knowing the services, knowing what the services can do for us in certain use cases and how they interact with other services, in this case CloudTrail and S3. All right, so that's a look at a few scenarios and our visualization exercises, and we'll continue this in each section of the course. So with that, I'm going to end this lesson. Thank you for joining me, Cloud Gurus. I will see you in the next lesson.